SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	S.PRATEEKSHA	**Reg. No.:**	192572137
Date:		**Duration:**	60 minutes

Code of Conduct

I _____P.AKSHAYA-192525190_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Employee performance evaluations and productivity analytics system

Annexure A – Architecture Design Assignment

1. System Requirements Analysis

System Objectives

The main objective of the Employee performance evaluations and productivity analytics system is to assist travelers in planning and enjoying their trips by providing personalized recommendations based on their interests, preferences, location, budget, and travel history. The system acts as a digital travel assistant that helps tourists discover attractions, restaurants, hotels, transportation services, and local events in real time.

The specific objectives of the system are:

- Provide personalized travel recommendations to users.
- Offer location-based tourist guidance and navigation.
- Help users plan trips efficiently.
- Deliver real-time information about tourist attractions, weather, events, and transportation.
- Improve the overall travel experience through intelligent recommendations.
- Support seamless interaction between tourists and tourism service providers.

Functional Requirements

The system should perform the following functions:

1. User Registration and Login
    - Users can create accounts and securely log in.
    - Support profile management and preference settings.
2. User Profile Management
    - Store traveler preferences such as interests, budget, language, and travel style.
    - Maintain travel history for future recommendations.
3. Recommendation Engine
    - Generate personalized suggestions for attractions, hotels, restaurants, and activities.
    - Recommend destinations based on user behavior and preferences.
4. Tourist Information Management
    - Provide details about tourist attractions, timings, ratings, and reviews.
    - Display maps and navigation information.
5. Real-Time Location Services
    - Detect user location using GPS.
    - Suggest nearby attractions and services.
6. Trip Planning and Itinerary Generation
    - Create customized travel plans.
    - Allow users to modify and save itineraries.
7. Booking Integration
    - Connect with external hotel, transport, and ticket booking services.
    - Display booking status and confirmations.
8. Reviews and Ratings
    - Allow users to submit reviews and ratings.
    - Use feedback to improve recommendations.
9. Notifications and Alerts
    - Send recommendations, reminders, weather alerts, and travel updates.
10. Administration Module
    - Manage users, tourist information, content, and system monitoring.

Non-Functional Requirements

The non-functional requirements significantly influence the software architecture.

- The system must support thousands of concurrent users.
- Response time for recommendations should be minimal.
- The system should be available 24/7.
- User data must be protected using secure authentication and encryption.
- The architecture should support future expansion.
- The system should be easy to maintain and update.
- The application should work across web and mobile platforms.
- Data accuracy and consistency should be maintained.

Key Quality Attributes

Scalability

The system should accommodate increasing numbers of users, tourist locations, and travel services without performance degradation.

Maintainability

The architecture should allow easy modification, testing, and deployment of new features.

Reliability

The system must consistently provide accurate recommendations and services without failures.

Availability

The application should remain operational even during peak tourism seasons.

Performance

Recommendations and location-based services should be generated quickly with low latency.

Security

User information, payment details, and travel records must be protected from unauthorized access.

---

2. Selection of Suitable Software Architecture

Chosen Architecture: Hybrid Microservices Architecture with MVC-Based Client Applications

The most suitable architecture for the Smart Tourist Guide and Personalized Travel Recommendation System is a Hybrid Architecture that combines:

- MVC Architecture for Web and Mobile User Interfaces
- Microservices Architecture for Backend Services

Justification

This system involves several independent functionalities such as:

- User management
- Recommendation generation
- Location tracking
- Booking integration
- Notifications
- Review management

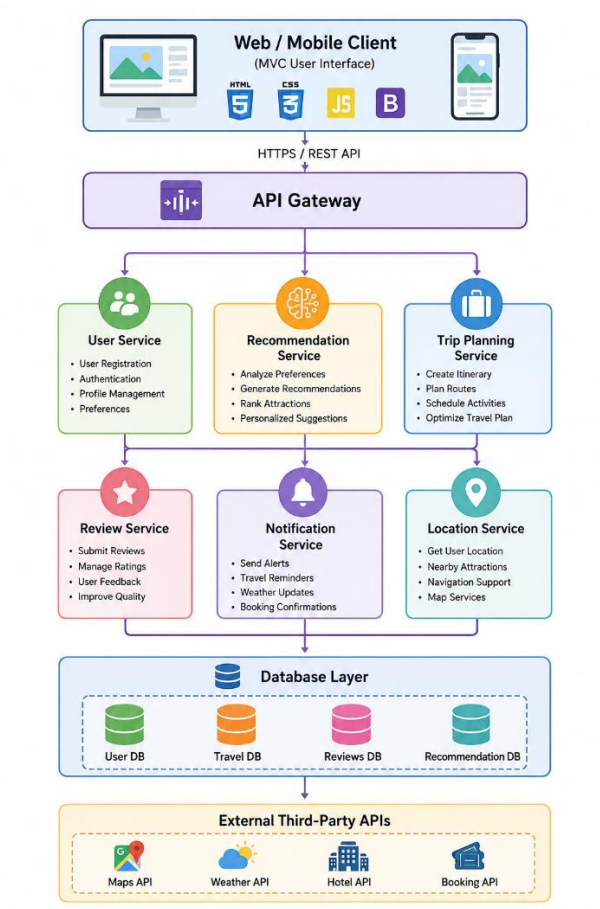
Each functionality can be developed and deployed independently using microservices. This improves scalability, maintainability, and reliability.

The MVC architecture on the client side separates the user interface, business logic, and data handling, making the application easier to manage and update.

Since the system interacts with multiple external services such as maps, weather APIs, and booking platforms, microservices provide greater flexibility and integration capabilities.

Therefore, a Hybrid Architecture offers the best balance between performance, scalability, maintainability, and future expansion.

3. Software Architecture Diagram



4. Components and Their Interactions

1. Web and Mobile Client

This component provides the user interface through which tourists interact with the system. Users can search destinations, view recommendations, create itineraries, and make bookings.

Responsibilities

- User interaction
- Display recommendations
- Collect user preferences
- Show maps and travel information

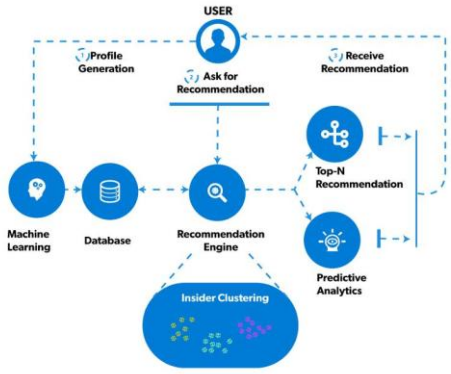
---

2. API Gateway

The API Gateway acts as a single entry point for all client requests.

Responsibilities

- Request routing
- Authentication verification
- Load balancing
- API management



3. User Service

This service manages user accounts and profile information.

Responsibilities

- Registration
- Login
- Profile management
- Preference storage

---

4. Recommendation Service

This is the core intelligence component of the system.

Responsibilities

- Analyze user preferences
- Process travel history
- Generate personalized recommendations
- Rank tourist attractions

---

5. Trip Planning Service

This service creates customized travel itineraries.

Responsibilities

- Schedule attractions
- Optimize routes

- Generate travel plans

---

6. Location Service

This service processes GPS and map information.

Responsibilities

- Track user location
- Find nearby attractions
- Provide navigation support

---

7. Review Service

This service handles user feedback.

Responsibilities

- Collect ratings
- Manage reviews
- Improve recommendation quality

---

8. Notification Service

This service delivers real-time alerts.

Responsibilities

- Travel reminders
- Weather alerts
- Booking notifications
- Promotional recommendations

---

9. Database Layer

The database layer stores all system data.

Responsibilities

- User information
- Tourist attraction details
- Reviews and ratings
- Recommendation history
- Travel plans

---

10. External APIs

Third-party services provide additional functionality.

Examples

- Google Maps API
- Weather API
- Hotel Booking API
- Transport Booking API

Overall Workflow

1. The user logs into the system.
2. User preferences and travel history are retrieved.
3. The Recommendation Service analyzes the user profile.
4. Location Service identifies the user's current position.
5. Recommendations are generated and displayed.
6. The user selects attractions and creates an itinerary.
7. Booking requests are sent to external booking services.
8. Notifications and travel updates are delivered in real time.
9. Users provide ratings and reviews after visiting locations.
10. The system updates recommendation models using collected feedback.

---

5. Quality Attribute Evaluation

Scalability

The architecture supports scalability through independent microservices.

- Services can scale individually.
- Load balancing distributes traffic efficiently.
- Additional servers can be added when demand increases.

Security

Security is ensured through multiple mechanisms.

- Secure authentication and authorization.
- HTTPS communication.
- Data encryption.
- Role-based access control.
- API Gateway security filtering.

Performance

The system delivers high performance because:

- Microservices operate independently.
- Frequently accessed data can be cached.
- Recommendation processing is distributed.
- API Gateway optimizes request routing.

Reliability

Reliability is improved through:

- Service isolation.
- Database backups.
- Error handling and recovery mechanisms.
- Continuous monitoring.

Maintainability

Maintainability is enhanced because:

- Services are loosely coupled.
- Modules can be updated independently.
- Easier testing and debugging.
- Clear separation of responsibilities.

Availability

The system achieves high availability through:

- Cloud deployment.
- Redundant service instances.
- Load balancing.
- Automatic failover mechanisms.

---

6. Justification of Architectural Decisions

Rationale Behind Architectural Choices

The Hybrid Microservices Architecture was selected because the system contains multiple independent functionalities that require different levels of scalability and maintenance.

Microservices allow each service to operate independently while MVC provides a structured user interface design. This combination creates a flexible and robust solution for tourism applications.

Trade-Offs Considered

Advantages

- High scalability
- Better fault isolation
- Easier maintenance
- Faster deployment of new features
- Improved flexibility

Disadvantages

- Higher development complexity
- More infrastructure requirements
- Increased monitoring and management effort

Despite these challenges, the benefits significantly outweigh the drawbacks for a large-scale tourism platform.

Support for Future Enhancements

The proposed architecture supports future growth in several ways:

- New recommendation algorithms can be added without affecting other services.
- Additional tourism providers can be integrated through APIs.
- AI and machine learning modules can be introduced for smarter recommendations.
- Multilingual support can be added easily.
- New services such as virtual tours, augmented reality guidance, and chatbot assistance can be incorporated without redesigning the entire system.

Conclusion

The Smart Tourist Guide and Personalized Travel Recommendation System is best implemented using a Hybrid Architecture combining MVC and Microservices. This architecture satisfies all major functional and non-functional requirements while ensuring scalability, security, reliability, maintainability, availability, and high performance. It also provides a strong foundation for future expansion and integration with emerging tourism technologies.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope;	Justification is weak or absent; report lacks organization and technical clarity.

	presents a well-organized, professional report.		report needs improvement.	

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25